

Telemetría

Enlace al repositorio de GitHub del sistema de telemetría: [Telemetry System](#)

Diseño de clases

El sistema está diseñado para integrarse desde distintos lenguajes y motores a través de una API C estable, manteniendo internamente una arquitectura orientada a objetos. La clase central es el **tracker**, responsable de recibir eventos, gestionar su cola en memoria, serializarlos y delegar su persistencia.

Instancias de tracker y configuraciones independientes

La API permite crear múltiples instancias de **tracker** mediante `CreateTracker(serializationFormat, persistenceMethod, eventQueuePolicy, pathEventFile)`. Cada instancia se inicializa con su propia combinación de:

- formato de serialización
- backend de persistencia
- política de cola
- ruta de salida (cuando aplica a persistencia en fichero).

Esto permite, por ejemplo, que un subsistema del juego persista en un fichero JSON específico y otro subsistema utilice una configuración diferente sin compartir estado interno entre trackers.

El metodo concreto para construir un tracker está pensado para las funcionalidades actuales y conforme se agregue funcionalidad se podrían agregar más sobrecargas o dar soporte a la inicializacion a partir de un fichero de configuracion.

EventData, ownership de memoria y AttributeData

Con el objetivo de que el sistema de telemetría tenga el menor impacto posible en el juego, es este sistema el que se encarga de crear y destruir la memoria asociada a los eventos.

El evento se modela con un bloque contiguo para la cabecera (`eventTypeID`, `timestamp`, `attributeCount`) y un array dinámico de atributos (`attributes`). Cada atributo contiene:

- identificador de atributo (*attributeNameID*),
- identificador de tipo (*attributeTypeID*),
- valor (*AttributeValue*, unión de tipos soportados).

La creación y destrucción explícita se expone por API (`CreateEvent` y `DestroyEvent`) para controlar el coste de memoria y mantener un contrato ABI claro entre lenguajes.

El ownership sigue esta regla principal: una vez invocado `TrackEvent`, la librería pasa a ser responsable del ciclo de vida del evento, tanto en éxito como en error. Por tanto, el código cliente no debe volver a acceder ni liberar ese puntero tras la llamada.

Pseudocódigo de uso recomendado:

```
event = CreateEvent(numAttributes=2)
if event == null:
    return ERROR_OOM

event.eventTypeID = PLAYER_JUMP
event.timestamp = now()

event.attributes[0].attributeNameID = ATTR_HEIGHT
event.attributes[0].attributeTypeID = Float
event.attributes[0].value.f = 1.25

event.attributes[1].attributeNameID = ATTR_SURFACE
event.attributes[1].attributeTypeID = FixedStr
event.attributes[1].value.FixedStr = "metal"

result = TrackEvent(tracker, event)

# No llamar a DestroyEvent(event) despues de TrackEvent.
# El tracker ya es propietario del evento.
```

Este diseño deja preparada una evolución futura hacia pools de objetos (eventos y/o arrays de atributos), ya que el ownership está centralizado en la librería y no disperso en el código cliente.

Además con este diseño solo tenemos 2 llamadas entre lenguajes por evento (create + track), con lo cual se reduce lo máximo posible el coste de pasar de un lenguaje a otro, además de que como la memoria se está rellenando directamente, se evitan copias innecesarias de los eventos.

Como detalle importante, para utilizar este sistema desde otros lenguajes es necesario declarar los Structs de EventData y AttributeData con un layout de memoria exactamente igual que al que hay en C++, para garantizar que los datos se pasan correctamente entre lenguajes

Gestión de cola de eventos y políticas actuales

Actualmente la política implementada es una cola circular en array (`CIRCULAR_ARRAY`) con capacidad fija interna. El comportamiento operativo es:

- inserción normal cuando hay hueco (`TrackEvent` devuelve `0`),
- sobrescritura del evento más antiguo cuando la cola está llena (`TrackEvent` devuelve `1`),
- fallo de tracking (`-1`) cuando no se puede procesar el evento.

La operación de vaciado (`Flush`) serializa los eventos pendientes a un único chunk y realiza una única llamada de persistencia para ese bloque. Si algún evento no se puede serializar, se descarta y se continúa con el resto; este caso se reporta con retorno `-2` si la persistencia global sí termina correctamente. Si la persistencia falla, se devuelve `-1`.

Pseudocódigo simplificado de la política de cola:

```
if queue.isFull():
    destroy(queue.head)
    queue.head = newEvent
    advanceHead()
    return OVERWRITE_OLDEST
else:
    queue.pushBack(newEvent)
    return INSERTED
```

Además, al cerrar el tracker (`CloseTracker`) se intenta vaciar la cola y cerrar correctamente el stream serializado (por ejemplo, cierre del array JSON), minimizando impacto en tiempo de ejecución del juego y evitando propagar errores en fase de cierre.

Es importante destacar que el sistema no valida los eventos que recibe en `TrackEvent`, sino que asume que ya están validados, y simplemente se encarga de gestionar su ciclo de vida y serializarlos y persistirlos cuando se hace la llamada a flush

Extensión con nuevos formatos de serialización y persistencia

La extensibilidad se basa en interfaces:

- serialización: implementar `ISerializer`,
- persistencia: implementar `IPersistence`.

Para registrar una nueva implementación en el tracker, el flujo esperado es:

1. añadir la nueva clase concreta (por ejemplo, `BinarySerializer` o `FirebasePersistence`),
2. ampliar los enums de configuración con el nuevo identificador,
3. actualizar la selección en inicialización (`initSerializer` o `initPersistence`) para construir la implementación adecuada,
4. mantener el contrato de error/cierre (`serialize`, `persist`, `close`) para conservar el comportamiento del tracker.

Pseudocódigo de extensión:

```
switch serializationFormat:
    case JSON: serializer = new JsonSerializer()
    case BINARY: serializer = new BinarySerializer()
    default: return INIT_ERROR

switch persistenceMethod:
    case FILE: persistence = new FilePersistence(path)
    case HTTP: persistence = new HttpPersistence(endpoint)
    default: return INIT_ERROR
```

Con este enfoque, el sistema mantiene una API estable hacia fuera, mientras permite implementar más funcionalidad de manera sencilla y retrocompatible.